BUILD REPORT

Making the Classroom Scene Interactive

Third-person control, seating, dialogue and hinged parts, built from static geometry in Unity

PROJECT	C:\Users\user\school
ENGINE	Unity 6000.6.0f1, Universal Render Pipeline 17.6.0
INPUT	Input System 1.20.0, polled each frame
SCENE	Assets/Scenes/SampleScene.unity
SCRIPTS	7 files, 534 lines of C#
INTERACTABLES	43 wired: 33 seats, 5 students, 5 hinged parts

27 October 2026

1. Starting point

The scene opened as static geometry. A classroom shell of 12 m by 12 m, 347 furniture objects under a `Furniture` root, 5 Mixamo student rigs, an imported player character, one door, four windows, a whiteboard and a projector screen. Everything was a mesh with a material. Nothing listened to a keypress, nothing moved, and the camera sat at its default position looking at a wall.

The goal was a third-person character that walks through the building, sits on any chair, talks to any student, and opens the door and the window sashes. Every step below was verified by rendering frames from inside the scene, not by reading the console.

2. The seven scripts

Script	Lines	Job
`Interactable.cs`	16	Abstract contract: `Prompt`, `CanInteract()`, `Interact()`
`PlayerController.cs`	183	Movement, gravity, jump, target detection, prompt banner
`CameraFollow.cs`	65	Third-person orbit camera with cursor lock
`SitInteractable.cs`	41	Seat behaviour, sit/stand request
`TalkInteractable.cs`	57	Dialogue lines and speech bubble
`PivotInteractable.cs`	48	Hinged rotation for door and windows
`SeatedPoseFit.cs`	124	Two-bone IK that plants the feet while seated

3. Step 1 — Standing the player up

The `Player` object carries exactly three components: `Transform`, `CharacterController`, `PlayerController`. No Rigidbody. A `CharacterController` is kinematic, so it slides along walls and climbs small steps without the mass, drag and constraint tuning a dynamic capsule would need.

Controller settings came from the geometry of the room: height 1.8 m, radius from the auto-fit capsule, centre at y = 0.90, slope limit 45°, step offset 0.3 m, skin width 0.08 m.

The visible character is the `Visual` child at local position zero, scale 1:1, feet at the pivot, with `PlayerAnimator.controller` on the rig root beneath it. That controller has two parameters and three states:

- `Speed` (float) — `0` Idle, `1` Walk
- `Seated` (bool) — switches to the sitting clip
- States: `Idle` → `Idle_InPlace`, `Walk` → `Walk_InPlace`, `Sitting` → `Sitting_InPlace`

All three clips are in-place, so the controller owns world position. Mixing in-place clips with root motion would have fought the `CharacterController`.

Locomotion

Input is read directly from `Keyboard.current` each frame, eight bindings: WASD plus the arrow keys. The input vector is rebuilt on the camera's basis with Y flattened, then normalised, which kills the classic diagonal speed-up (1.41× on two keys).

Body rotation uses `Quaternion.Slerp` at 12/s toward the travel direction, so the character turns into the walk instead of snapping. Gravity is a manual accumulator at −18 m/s². While grounded it resets to −2 rather than 0, a small downward bias that keeps the capsule glued to slopes and stairs. Jump fires at 4.5 m/s only when grounded.

The animation parameter is not the input. It is `controller.velocity` flattened and divided by `moveSpeed`, so a character pushed against a wall stands still with idle playing instead of walking in place.

4. Step 2 — Third-person camera

`CameraFollow` sits on `Main Camera` and re-derives its target at runtime, which is why `target` shows as `None` in the Inspector and the view still works.

The rig is a fixed arm: 3.2 m behind, 1.55 m above the player's feet. Mouse delta drives yaw and pitch at 2.2 × 0.1 sensitivity, with pitch clamped between −30° and 65° so the camera cannot roll under the floor or flip over the head. Positioning happens in `LateUpdate` with exponential smoothing, `1 − exp(−12·dt)`, which is framerate independent. A plain `Lerp` with a fixed factor would feel slower on a slower machine.

Cursor handling: locked on `Start`, `Esc` releases it, left click grabs it back. That last part matters in the editor, where losing the cursor to a locked state is otherwise a restart.

Movement stays camera-relative, so W always walks away from the camera regardless of where the character faces.

5. Step 3 — The interaction framework

The whole system hangs on four lines of contract:

```
public abstract class Interactable : MonoBehaviour
{
    public virtual string Prompt { get; }
    public virtual bool CanInteract() { return true; }
    public abstract void Interact();
}
```

Anything that inherits it becomes a candidate: a chair, a student, a door. `PlayerController` finds candidates with `Physics.OverlapSphere` at chest height (player + 0.9 m), radius 2.4 m, once per frame. The nearest one by squared distance wins. Colliders are resolved upward with `GetComponentInParent`, so the seat mesh owns its own behaviour instead of the chair group needing a component on every child.

`E` fires `Interact()` on the current target. The prompt renders through `OnGUI` at 60% of screen height: a 360 × 34 box, 18 pt, white on 55% black.

Two deliberate simplifications: no raycast, and no trigger volumes. Sphere overlap is three lines and cannot miss something the player is standing inside, which happens constantly with chairs. The trade-off is that it also detects objects behind the player. At 2.4 m in a room this size, that never produced a wrong prompt.

6. Step 4 — Sitting on 33 seats

Each of the 32 chairs and the teacher chair has a `_Seat` child mesh carrying `SitInteractable` and a `BoxCollider`. One example: `Chair_D0_0A_Seat` anchored at (−4.37, 0.45, −2.54).

`SitAt` disables the `CharacterController` (so physics stops arguing with the snap), places the player on the seat, sets the facing to 180°, clears vertical velocity and flips the `Seated` flag. `StandUp` re-enables the controller and lifts the player 0.05 m first, because re-enabling the capsule exactly where the chair mesh already is produces a depenetration shove.

The prompt is state aware: "Press E to sit down" or "Press E to stand up", read from the player, not from the seat.

The pose problem, and the IK that fixed it

The sitting clip was authored for a different hip height. Once the player model was swapped in, the feet hung in the air and the hips floated above the cushion. Scaling the rig was not an option; the rest of the body would have been wrong.

`SeatedPoseFit` solves it with two-bone IK in `LateUpdate`:

1. Measure thigh (`mixamorig:LeftUpLeg` → `LeftLeg`) and shin (`LeftLeg` → `LeftFoot`).
2. Pick an ankle target at `player.y - 0.08 + 0.1544` (the 0.1544 m is sole-to-ankle for this rig), pushed 0.26 m forward.
3. Clamp the hip-to-ankle distance to leg length minus 1 mm so the knee never hyperextends.
4. Rebuild the knee position with the law of cosines, keeping the existing knee direction as the pole vector.
5. Slerp hip and knee corrections, then flatten the foot with a signed pitch from the toe bone.

Every correction is blended by a 0→1 ramp at 8/s, so sitting down and standing up animate instead of popping. The whole rig also sinks 0.10 m while seated, and that is what puts the hips on the cushion.

7. Step 5 — Talking to the five students

`TalkInteractable` holds a speaker name, an array of lines, and a display duration. Five students are wired with their own script and three lines each:

Student	Controller	First line
Ana	`AC_Ana_Idle`	"Did you finish the homework for today?"
Bruno	`AC_Bruno_Phone`	"Sorry, I'm on the phone."
Carla	`AC_Carla_Idle`	"Do you have a spare pen? Mine just died."
Diego	`AC_Diego_Idle`	"Psst... is there a test today?"
Elena	`AC_Elena_Idle`	"Hi! Is this seat taken?"

Each press advances one line and resets a 5 second timer; the bubble draws at 74% of screen height with a 460 × 62 dark panel. A static `active` reference gates rendering, so walking between two students can never stack two bubbles.

All five carry a `CapsuleCollider` so the overlap sphere finds them, and a looping idle animator so they are not statues.

8. Step 6 — Door and window sashes

One component covers both cases. `PivotInteractable` records the closed rotation in `Awake`, multiplies it by `openEuler` once, and rotates toward whichever target is current with `Quaternion.RotateTowards` at `speed × 90` = 270°/s. It snaps and clears the animating flag within 0.1° of target. The prompt swaps between open and close.

Wired to five parts: `DoorPivot` (+90°), `Win1_Sash` (+90°), `Win2_Sash` (−90°), `Win3_Sash` (−90°), `Win4_Sash` (+90°). The signs mirror the hinge side; a sash on the left of the wall swings out the other way.

Because the closed rotation is captured at startup rather than hard-coded, the same component can be dropped on any hinge without edits.

9. Step 7 — Wiring the scene

Root object	Children	Role
`Building`	117	Shell, roof, corridor, bands, door
`Furniture`	347	Desks, chairs, seats
`Lighting`	18	Ceiling fixtures
`Boards`	11	Whiteboard, screen
`Students`	5	Rigs plus talk triggers
`Environment`	1	Ground plane
`Player`	1	Character plus visual
`Interactables`	0	Empty group, left over

No prefabs were used for the interactables; each seat child was created under its own chair and given the script, collider and anchor values directly. That kept the scene flat and easy to inspect, at the cost of repeated values across 33 seats.

10. Step 8 — Verification

Checks that caught problems:

Check	What it found
Play-mode run	Prompts only appeared when facing a chair, not from behind
Rendered frame of seated player	Feet floating, hips above cushion → led to `SeatedPoseFit`
Rendered frame at the door	Sash rotated into the frame instead of away from it
`Physics.Raycast` from outside	Ground collider invisible until `Physics.SyncTransforms()`
Walk test across the floor edge	2 cm slab step silently blocked without `stepOffset`

Edit-mode limitation worth knowing: animators do not evaluate in edit mode, so a seated pose can only be judged from a play-mode capture or a manually posed rig. Three of the fixes above came from looking at a picture, not at a log.

11. What is interactive now

- Walk, turn, jump, gravity, camera-relative steering
- Sit and stand on 33 seats, with the character re-posed to match the cushion
- Talk to 5 students, 3 rotating lines each, 5 second bubbles
- Open and close the door plus 4 window sashes
- Third-person orbit camera with cursor lock

12. Known gaps

No audio. The scene holds zero `AudioSource` components, so footsteps, door hinges and speech are all silent.

The camera arm is fixed length. Standing with a wall behind pulls the camera inside the wall mesh; a spring-arm raycast would fix it and has not been added.

Prompts are immediate-mode `GUI`. It works, but it bypasses the Canvas and cannot be styled or scaled with the UI system.

Targets are not highlighted. The only feedback that something is interactable is the prompt text.

Students are stationary. They loop idle clips and talk when clicked; nothing walks a path or turns to face the player.

Keys are hard-coded in `PlayerController` and `CameraFollow`. No rebinding, no gamepad, no touch.

No save or load, and no objective or progression layer. The scene is a sandbox.

13. Controls

Input	Action
`W A S D` / arrows	Move, camera relative
Mouse	Orbit camera
`Space`	Jump, or stand up when seated
`E`	Interact / talk / open / sit / stand
`Esc`	Release cursor
Left click	Recapture cursor